

Tercer Objetivo: Identificar las posibles diferencias metabólicas mediante el análisis comparativo del flujo metabólico de los tres mutantes sólidos.

1. Mapear los reaction KEGG IDs a map KEGG IDs con sus respectivas rutas metabólicas

Usando el comando **tmux attach-session -t 0** accedemos a la sesión **0** de tmux en la cual podemos ejecutar comandos directamente en el servidor, los cuales son inmunes a desconexiones entre la máquina local y el servidor. En la sesión de tmux para construir el script de mapeo se usó la herramienta nano, la cual ya estaba instalada en el servidor.

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux attach-session -t 0  
[detached (from session 0)]
```

Con el comando **nano ReacToPath12.py** se creó el archivo de python para mapear los IDs con el siguiente algoritmo:

NOTA: Los errores 404 indican que el reaction ID no existe en KEGG. Por otro lado, si aparece un error 400, el formato del reaction ID es inválido (formatos como RK0002 no son estándar del formato reaction de KEGG).

NOTA: con grandes cantidades de datos se sugiere aumentar el retraso entre cada búsqueda de la API de KEGG, pues es limitada. Para instalar el paquete **requests** se puede ejecutar el comando **pip install requests**.

```
#!/usr/bin/env python3
```

```
"""
```

Mapeador de KEGG reaction ID a pathway mapper ID con seguimiento de IDs encontrados. Mantiene la fiabilidad de las limitaciones del KEGG API.

```
"""
```

```
import argparse
```

```
import csv
```

```
import json
```

```
import re
```

```
import sys
```

```
import time
```

```
from collections import defaultdict
```

```
from pathlib import Path
```

```
from typing import Dict, List, Optional, Tuple
```

```
from urllib.parse import quote
```

```
import requests
```

```
from requests.adapters import HTTPAdapter
```

```
from urllib3.util.retry import Retry
```

```
class KEGGMultiMapper:
```

```
    """Cliente de KEGG API con manejo de errores."""
```

```
    # Constantes de Class-level
```

```
    KEGG_REST_URL = "https://rest.kegg.jp/"
```

```
    KEGG_WEB_URL = "https://www.kegg.jp/entry/"
```

```
    REACTION_PATTERN = re.compile(r'R\d{5}|RK\d{4}')
```

```
    COMPOUND_PATTERN = re.compile(r'C\d{5}')
```

```
    RELATED_RN_PATTERN = re.compile(r'R\d{5}')
```

```
    SYNTHETIC_PATTERNS = ['Ex_', 'Biomass', 'sink_', 'DM_', 'SK_', 'BIOMASS']
```

```
    def __init__(self, delay: float = 1.0, timeout: int = 30, max_retries: int = 3, state_file: str  
= "kegg_mapper_state.json"):
```

```
        """Inicializa con parámetros configurables."""
```

```
        self.delay = delay
```

```
        self.timeout = timeout
```

```
        self.max_retries = max_retries
```

```
        self.state_file = Path(state_file)
```

```
        self._init_session()
```

```
        self.cache = {}
```

```
        self.equivalence_cache = {}
```

```
        self.pathway_cache = {}
```

```
        self.found_ids = self._load_state()
```

```
    def _init_session(self) -> None:
```

```
        """Configura la sesión de solicitudes con estrategia de reintento."""
```

```
        self.session = requests.Session()
```

```
        retry_strategy = Retry(  
            total=self.max_retries,
```

```
            backoff_factor=1,
```

```
            status_forcelist=[408, 429, 500, 502, 503, 504],
```

```
            respect_retry_after_header=True
```

```
        )
```

```
        adapter = HTTPAdapter(max_retries=retry_strategy)
```

```
        self.session.mount("https://", adapter)
```

```
        self.session.headers.update({  
            'Accept': 'text/plain',  
            'User-Agent': 'KEGG-Mapper/2.2 (Python)'  
        })
```

```

def _load_state(self) -> Dict:
    """Carga IDs previamente encontrados del archivo de estado."""
    try:
        if self.state_file.exists():
            with self.state_file.open('r') as f:
                return json.load(f)
    except Exception as e:
        print(f"Warning: Could not load state file: {e}", file=sys.stderr)
    return {}

```

```

def _save_state(self) -> None:
    """Guarda el estado actual de los IDs encontrados en el archivo."""
    try:
        with self.state_file.open('w') as f:
            json.dump(self.found_ids, f, indent=2)
    except Exception as e:
        print(f"Warning: Could not save state file: {e}", file=sys.stderr)

```

```

def _update_state(self, reaction_id: str, info: Dict) -> None:
    """Actualiza el estado con un nuevo ID encontrado - solo si es un Found ID (no KEGG
    Verified, Via Equiv, etc.)."""
    if info.get('found'):
        self.found_ids[reaction_id] = {
            'pathways': info.get('pathways', []),
            'status': info.get('api_status', ''),
            'timestamp': time.strftime("%Y-%m-%d %H:%M:%S")
        }
        self._save_state()

```

```

def clean_reaction_id(self, reaction_id: str) -> str:
    """Normaliza y valida el formato de KEGG reaction ID."""
    reaction_id = reaction_id.strip().upper()

    if reaction_id.startswith('R_'):
        reaction_id = reaction_id[2:]
    elif reaction_id.startswith('RN:'):
        reaction_id = reaction_id[3:]

    if not (self.REACTION_PATTERN.fullmatch(reaction_id) or
            reaction_id.startswith('RK') and len(reaction_id) == 6):
        raise ValueError(f"Invalid KEGG reaction ID format: {reaction_id}")

    return reaction_id

```

```

def query_kegg_api(self, endpoint: str, query: str) -> str:
    """Query genérico para la KEGG API con manejo de errores."""
    cache_key = f"{endpoint}/{query}"

    if cache_key in self.cache:
        return self.cache[cache_key]

    url = f"{self.KEGG_REST_URL}{endpoint}/{quote(query)}"

    try:
        response = self.session.get(url, timeout=self.timeout)

        if response.status_code == 403:
            time.sleep(5)
            return ""
        elif response.status_code == 400:
            return ""
        elif response.status_code == 404:
            return ""

        response.raise_for_status()

        result = response.text
        self.cache[cache_key] = result
        return result

    except requests.exceptions.HTTPError as e:
        if hasattr(e, 'response') and e.response.status_code in (400, 403, 404):
            return ""
        raise ConnectionError(f"KEGG API request failed: {e}")
    except requests.exceptions.RequestException as e:
        raise ConnectionError(f"Network error: {e}")

def verify_reaction_existence(self, reaction_id: str) -> Dict:
    """Verificación comprensiva de la existencia de reacciones en KEGG."""
    result = {
        'exists': False,
        'type': None,
        'location': None,
        'notes': []
    }

    try:
        clean_id = self.clean_reaction_id(reaction_id)

```

```

api_result = self.query_kegg_api("get", clean_id)

if api_result:
    result.update({
        'exists': True,
        'type': 'standard_reaction',
        'location': 'REST_API'
    })
    return result

web_result = self._verify_via_kegg_website(clean_id)
if web_result['exists']:
    result.update(web_result)

return result

except Exception as e:
    result['notes'].append(f"Verification error: {str(e)}")
    return result

def _verify_via_kegg_website(self, reaction_id: str) -> Dict:
    """Evalúa la existencia de reacciones vía KEGG website."""
    result = {
        'exists': False,
        'type': None,
        'location': 'KEGG_Website',
        'notes': []
    }

    try:
        url = f"{self.KEGG_WEB_URL}{reaction_id}"
        response = self.session.get(url, timeout=self.timeout)

        if response.status_code == 200 and "No such data" not in response.text:
            result['exists'] = True
            content = response.text

            if "Reaction" in content:
                result['type'] = 'reaction'
            elif "Compound" in content:
                result['type'] = 'compound'
                result['notes'].append('This is actually a compound ID')
            elif "Orthology" in content:
                result['type'] = 'orthology'

```

```

        elif "Module" in content:
            result['type'] = 'module'
        else:
            result['type'] = 'unknown'

    return result
except Exception as e:
    result['notes'].append(f"Website verification error: {str(e)}")
return result

def get_pathways_for_reaction(self, reaction_id: str) -> List[Tuple[str, str]]:
    """Obtención de pathways para una reacción usando múltiples estrategias."""
    try:
        clean_id = self.clean_reaction_id(reaction_id)

        strategies = [
            self._get_pathways_via_rest,
            self._get_pathways_via_link,
            self._get_pathways_via_equivalents
        ]

        for strategy in strategies:
            try:
                pathways = strategy(clean_id)
                if pathways:
                    return pathways
            except Exception:
                continue

        return []
    except ValueError:
        return []
    except ConnectionError:
        return []

def _get_pathways_via_rest(self, reaction_id: str) -> List[Tuple[str, str]]:
    """Obtención de pathways a través de REST API."""
    formats_to_try = [reaction_id, f"rn:{reaction_id}", f"R_{reaction_id}"]

    for fmt_id in formats_to_try:
        result = self.query_kegg_api("get", fmt_id)
        if result:
            return self._parse_pathways_from_get(result)

```

```
return []
```

```
def _get_pathways_via_link(self, reaction_id: str) -> List[Tuple[str, str]]:
```

```
    """Búsqueda de pathway alternativo usando link API."""
```

```
    result = self.query_kegg_api("link/pathway", reaction_id)
```

```
    if not result:
```

```
        return []
```

```
    pathways = []
```

```
    for line in result.strip().split('\n'):
```

```
        if not line:
```

```
            continue
```

```
        parts = line.split('\t')
```

```
        if len(parts) == 2:
```

```
            map_id = parts[1].replace('path:', '')
```

```
            pathways.append((map_id, self.get_pathway_name(map_id)))
```

```
    return pathways
```

```
def _get_pathways_via_equivalents(self, reaction_id: str) -> List[Tuple[str, str]]:
```

```
    """Obtención de pathways vía equivalent reactions."""
```

```
    equivalents = self.check_reaction_equivalents(reaction_id)
```

```
    for eq in equivalents[:3]: # Limit to first 3 equivalents
```

```
        pathways = self._get_pathways_via_rest(eq)
```

```
        if pathways:
```

```
            return pathways
```

```
    return []
```

```
def _get_pathways_via_website(self, reaction_id: str) -> List[Tuple[str, str]]:
```

```
    """Intento de obtención de información de pathway vía website scraping."""
```

```
    try:
```

```
        url = f"{self.KEGG_WEB_URL}{reaction_id}"
```

```
        response = self.session.get(url, timeout=self.timeout)
```

```
        if response.status_code == 200:
```

```
            pathways = []
```

```
            # Búsqueda de pathway links en la página
```

```
            for line in response.text.split('\n'):
```

```
                if 'path:' in line and 'map' in line:
```

```
                    # Extraer los pathway IDs de los atributos href
```

```
                    matches = re.findall(r'href="/pathway/(map\d+)"', line)
```

```
                    for map_id in matches:
```

```
                        pathways.append((map_id, self.get_pathway_name(map_id)))
```

```
            return list(set(pathways))
```

```

except Exception as e:
    pass
return []

def get_pathway_name(self, map_id: str) -> str:
    """Obtener el pathway name de un map ID con el caché."""
    if map_id not in self.pathway_cache:
        result = self.query_kegg_api("get", map_id)
        if not result:
            self.pathway_cache[map_id] = ""
        else:
            first_line = result.split("\n")[0]
            if 'PATHWAY' in first_line:
                name = first_line.split('PATHWAY')[1].strip()
            else:
                name = first_line.split('\t')[-1].strip()
            self.pathway_cache[map_id] = name

    return self.pathway_cache[map_id]

def get_reaction_compounds(self, reaction_id: str) -> List[str]:
    """Obtención de compuestos involucrados en la reacción."""
    result = self.query_kegg_api("get", reaction_id)
    if not result:
        return []

    compounds = []
    for line in result.split("\n"):
        if line.startswith('EQUATION'):
            compounds.extend(self.COMPOUND_PATTERN.findall(line))

    return list(set(compounds))

def check_reaction_equivalents(self, reaction_id: str) -> List[str]:
    """Evalúa si una reacción tiene reacciones equivalentes en KEGG."""
    if reaction_id in self.equivalence_cache:
        return self.equivalence_cache[reaction_id]

    equivalents = []
    result = self.query_kegg_api("get", reaction_id)
    if result:
        for line in result.split("\n"):
            if line.startswith('RELATED_RN'):
                equivalents.extend(self.RELATED_RN_PATTERN.findall(line))

```



```

if not equivalents:
    compounds = self.get_reaction_compounds(reaction_id)
    for cpd in compounds[:3]: # Limit to first 3 compounds
        cpd_linked = self.query_kegg_api("link/rn", cpd)
        if cpd_linked:
            equivalents.extend(re.findall(r'rn:(R\d{5})', cpd_linked))

equivalents = list(set(equivalents))
self.equivalence_cache[reaction_id] = equivalents
return equivalents

def _parse_pathways_from_get(self, kegg_text: str) -> List[Tuple[str, str]]:
    """Parsea pathway information del KEGG API response."""
    pathways = []
    in_pathway = False

    for line in kegg_text.split('\n'):
        if line.startswith('PATHWAY'):
            in_pathway = True
            parts = re.split(r'\s{2,}', line[12:].strip())
            if len(parts) >= 2:
                pathways.append((parts[0], ' '.join(parts[1:])))
            elif line.startswith(' ') and in_pathway:
                parts = re.split(r'\s{2,}', line.strip())
                if len(parts) >= 2:
                    pathways.append((parts[0], ' '.join(parts[1:])))
            else:
                in_pathway = False

    return pathways

def get_reaction_info(self, reaction_id: str) -> Dict:
    """Mejora la reaction information con mejor detección de pathway."""
    # Evalúa si ya existe la reacción en found_ids
    if reaction_id in self.found_ids:
        cached_info = self.found_ids[reaction_id]
        return {
            'original_id': reaction_id,
            'clean_id': reaction_id,
            'found': True,
            'pathways': cached_info.get('pathways', []),
            'equivalents': [],
            'kegg_mapper_verified': True,

```

```

        'id_type': 'cached',
        'notes': ['Loaded from cached results'],
        'api_status': cached_info.get('status', 'cached')
    }

info = {
    'original_id': reaction_id,
    'clean_id': "",
    'found': False,
    'pathways': [],
    'equivalents': [],
    'kegg_mapper_verified': False,
    'id_type': 'unknown',
    'notes': [],
    'api_status': 'success'
}

try:
    # Salta reacciones sintéticas mencionadas
    if any(pattern in reaction_id for pattern in self.SYNTHETIC_PATTERNS):
        info['notes'].append("Synthetic reaction detected")
        info['api_status'] = 'synthetic_reaction'
        return info

    clean_id = self.clean_reaction_id(reaction_id)
    info['clean_id'] = clean_id

    verification = self.verify_reaction_existence(reaction_id)
    info['id_type'] = verification.get('type', 'unknown')

    if verification['type'] == 'compound':
        info['notes'].append("ID is actually a compound, not a reaction")
        info['api_status'] = 'compound_id'
        return info

    # Mejora en detección de pathway
    info['pathways'] = self.get_pathways_for_reaction(clean_id)

    if info['pathways']:
        info['found'] = True
        info['api_status'] = 'found'
    elif verification['exists']:
        info['kegg_mapper_verified'] = True
        info['notes'].append("Found in KEGG but no pathway data via API")

```

```

info['api_status'] = 'verified_no_pathway'

# Evaluación adicional para website pathways si falla la API
web_pathways = self._get_pathways_via_website(clean_id)
if web_pathways:
    info['pathways'] = web_pathways
    info['found'] = True
    info['notes'].append("Found pathways via website scraping")

if not info['pathways']:
    info['equivalents'] = self.check_reaction_equivalents(clean_id)
    if info['equivalents']:
        info['notes'].append(f"Found {len(info['equivalents'])} equivalent reactions")
        info['api_status'] = 'found_via_equivalent'

# Evaluación adicional de pathways equivalentes vía website
for eq in info['equivalents'][:3]: # Limita a los 3 primeros equivalentes
    eq_pathways = self._get_pathways_via_website(eq)
    if eq_pathways:
        info['pathways'] = eq_pathways
        info['found'] = True
        info['notes'].append(f"Found pathways via equivalent {eq} website")
        break

# Actualiza el estado si se encuentra algo que valga almacenar
self._update_state(reaction_id, info)

return info

except ValueError as e:
    info['notes'].append(str(e))
    info['api_status'] = 'invalid_id'
    return info

except ConnectionError as e:
    info['notes'].append(f"API connection error: {str(e)}")
    info['api_status'] = 'api_error'
    return info

except Exception as e:
    info['notes'].append(f"Unexpected error: {str(e)}")
    info['api_status'] = 'error'
    return info

```

```
def process_csv(input_file: str, output_file: str, batch_size: int = 3, delay: float = 2.5,
state_file: str = "kegg_mapper_state.json") -> None:
```

```
    """Procesa el archivo CSV con KEGG mapping comprensivo."""
```

```
    mapper = KEGGMultiMapper(delay=delay, state_file=state_file)
```

```
    try:
```

```
        input_path = Path(input_file)
```

```
        with input_path.open('r') as infile:
```

```
            reader = csv.DictReader(infile)
```

```
            if 'RxnID' not in reader.fieldnames:
```

```
                raise ValueError("Input CSV must contain an 'RxnID' column")
```

```
            fieldnames = reader.fieldnames + [
```

```
                'Map_KEGG_IDs',
```

```
                'Pathway_Names',
```

```
                'KEGG_Status',
```

```
                'ID_Type',
```

```
                'Equivalent_Reactions',
```

```
                'Mapping_Notes',
```

```
                'API_Status'
```

```
            ]
```

```
            rows = list(reader)
```

```
            total_rows = len(rows)
```

```
        processed_rows = []
```

```
        stats = defaultdict(int)
```

```
        not_found_verified = []
```

```
        compound_ids = []
```

```
        synthetic_reactions = []
```

```
        print(f"\nProcessing {total_rows} reactions with enhanced mapping...")
```

```
        print(f"Loaded {len(mapper.found_ids)} previously found reactions from cache\n")
```

```
        for i, row in enumerate(rows, 1):
```

```
            reaction_id = row['RxnID']
```

```
            reaction_info = mapper.get_reaction_info(reaction_id)
```

```
            if reaction_info['api_status'] == 'synthetic_reaction':
```

```
                stats['synthetic'] += 1
```

```
                status = "SYNTHETIC"
```

```
                synthetic_reactions.append(reaction_id)
```

```
            elif reaction_info['found']:
```

```
                stats['success'] += 1
```

```
                status = "FOUND"
```

```

elif reaction_info['id_type'] == 'compound':
    stats['compound'] += 1
    status = "COMPOUND_ID"
    compound_ids.append(reaction_id)
elif reaction_info['kegg_mapper_verified']:
    stats['mapper_verified'] += 1
    status = "VERIFIED_IN_KEGG"
    not_found_verified.append(reaction_id)
elif reaction_info['equivalents']:
    stats['equivalent_found'] += 1
    status = "FOUND_VIA_EQUIVALENT"
else:
    stats['not_found'] += 1
    status = "NOT_FOUND"

row.update({
    'Map_KEGG_IDs': '; '.join(p[0] for p in reaction_info['pathways']),
    'Pathway_Names': '; '.join(p[1] for p in reaction_info['pathways']),
    'KEGG_Status': status,
    'ID_Type': reaction_info['id_type'],
    'Equivalent_Reactions': '; '.join(reaction_info['equivalents']),
    'Mapping_Notes': ' | '.join(reaction_info['notes']),
    'API_Status': reaction_info.get('api_status', 'unknown')
})
processed_rows.append(row)

if i % 5 == 0 or i == total_rows:
    progress = int(i / total_rows * 100)
    print(f"\rProgress: {progress}% | "
          f"Found: {stats['success']} | "
          f"KEGG Verified: {stats['mapper_verified']} | "
          f"Via Equiv: {stats['equivalent_found']} | "
          f"Compounds: {stats['compound']} | "
          f"Synthetic: {stats['synthetic']} | "
          f"Not Found: {stats['not_found']}", end="", flush=True)

if i % batch_size == 0 and reaction_id not in mapper.found_ids:
    time.sleep(delay)

output_path = Path(output_file)
with output_path.open('w', newline=") as outfile:
    writer = csv.DictWriter(outfile, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(processed_rows)

```

```
        generate_reports(output_path, stats, not_found_verified, compound_ids,
synthetic_reactions, processed_rows)
```

```
except Exception as e:
    print(f"\nFatal error: {str(e)}", file=sys.stderr)
    sys.exit(1)
```

```
def generate_reports(output_path: Path, stats: Dict, not_found_verified: List[str],
        compound_ids: List[str], synthetic_reactions: List[str], rows: List[Dict]) ->
```

```
None:
```

```
    """Generate detailed report files."""
```

```
    base_name = output_path.stem
```

```
    report_dir = output_path.parent
```

```
    summary_path = report_dir / f"{base_name}_summary.txt"
```

```
    with summary_path.open('w') as f:
```

```
        f.write("KEGG MAPPING SUMMARY REPORT\n")
```

```
        f.write("=====\n\n")
```

```
        f.write(f"Total reactions processed: {sum(stats.values())}\n")
```

```
        f.write(f"Successfully mapped: {stats['success']}\n")
```

```
        f.write(f"Verified in KEGG but no pathway data: {stats['mapper_verified']}\n")
```

```
        f.write(f"Mapped via equivalent reactions: {stats['equivalent_found']}\n")
```

```
        f.write(f"Compound IDs (not reactions): {stats['compound']}\n")
```

```
        f.write(f"Synthetic reactions: {stats['synthetic']}\n")
```

```
        f.write(f"Not found: {stats['not_found']}\n\n")
```

```
    if not_found_verified:
```

```
        f.write("\nReactions found in KEGG but not via API:\n")
```

```
        f.write("\n".join(not_found_verified))
```

```
    if compound_ids:
```

```
        f.write("\n\nThe following IDs are actually compounds:\n")
```

```
        f.write("\n".join(compound_ids))
```

```
    if synthetic_reactions:
```

```
        f.write("\n\nThe following IDs are synthetic reactions:\n")
```

```
        f.write("\n".join(synthetic_reactions))
```

```
not_found = [row for row in rows if row['KEGG_Status'] == "NOT_FOUND"]
```

```
if not_found:
```

```
    not_found_path = report_dir / f"{base_name}_not_found_detailed.csv"
```

```
    with not_found_path.open('w') as f:
```

```

        writer = csv.DictWriter(f, fieldnames=['RxnID', 'ID_Type', 'Mapping_Notes',
'API_Status'])
        writer.writeheader()
        writer.writerows([
            'RxnID': row['RxnID'],
            'ID_Type': row['ID_Type'],
            'Mapping_Notes': row['Mapping_Notes'],
            'API_Status': row['API_Status']
        ] for row in not_found])

```

```

print("\n\nProcessing complete!")
print(f"Results saved to {output_path}")
print(f"Summary report saved to {summary_path}")
if not_found:
    print(f"Detailed not found report saved to {not_found_path}")

```

def main() -> None:

```

    """Interfaz de línea de comando para KEGG mapper."""
    parser = argparse.ArgumentParser(
        description='Enhanced KEGG reaction to pathway mapper with robust error
handling',
        formatter_class=argparse.ArgumentDefaultsHelpFormatter
    )
    parser.add_argument('-i', '--input', required=True,
        help='Input CSV file containing RxnID column')
    parser.add_argument('-o', '--output', required=True,
        help='Output CSV file with mapping results')
    parser.add_argument('--batch-size', type=int, default=3,
        help='Number of API requests per batch (recommend 3-5)')
    parser.add_argument('--delay', type=float, default=2.5,
        help='Delay between batches in seconds (recommend 2.5-5)')
    parser.add_argument('--state-file', type=str, default="kegg_mapper_state.json",
        help='File to store found reaction mappings between runs')

    args = parser.parse_args()
    process_csv(args.input, args.output, args.batch_size, args.delay, args.state_file)

```

```

if __name__ == '__main__':
    main()

```

1. Clases ReactoPath12.py (Funciones centrales)

La función **__init__()** inicializa el cliente de KEGG API con:

Dos parámetros limitantes (**delay**, **max_retries**, una configuración de sesión con estrategia de reintento, sistema de almacenamiento caché (**cache**, **equivalence_cache**, **pathway_cache**) y mantenimiento de estado a través del **state_file**.

La función **_init_session()** configura la sesión HTTP con:

Reintentos automáticos para solicitudes fallidas (HTTP 408, 429, 500-504), implementación de un backoff (evita errores o mejora el uso de recursos) exponencial entre intentos, un HTTP User-Agent request header (es una cadena de texto que permite a los servidores y network peers (dispositivos que participan en una red, actuando como cliente y servidor, compartiendo recursos y cooperando con otros pares sin necesidad de un servidor central) identificar una aplicación, sistema operativo, proveedor o versión del requesting user agent) hecho a la medida. Asimismo, implementa un adaptador HTTPS con connection pooling (una técnica usada para mejorar el desempeño de una aplicación al reutilizar conexiones existentes entre bases de datos en vez de crear nuevas por cada solicitud).

Funciones de manejo de estado

La función **_load_state()**: carga las reacciones previamente encontradas y guardadas en un archivo JSON.

La función **_save_state()**: preserva el estado actual en el disco

La función **_update_state()**: actualiza el estado solo para reacciones encontradas exitosamente

Funciones centrales del API

La función **clean_reaction_id()**: normaliza y valida KEGG reaction IDs

La función **query_kegg_api()**: es el manejador genérico de solicitudes de API que implica el almacenamiento de solicitudes en caché, manejo de errores para códigos comunes de estado HTTP y codificación de URL para queries

Funciones de verificación

La función **verify_reaction_existence()**: implementa un método múltiple de verificación con evaluación primaria vía REST API, retracción a website scraping (búsqueda en página web) y detección de tipo (reaction/compound/orthology)

La función **_verify_via_kegg_website()**: verifica basada en la web cuando la API falla

Funciones de mapeo de Pathway

La función **get_pathways_for_reaction()**: implementa una multiestrategia para encontrar pathway con búsqueda directa de REST API, enlace del API query, búsqueda de reacciones Equivalent y retroceso a Website scraping. Respectivamente implementa la función **_get_pathways_via_rest()**, **_get_pathways_via_link()**, **_get_pathways_via_equivalents()** y **_get_pathways_via_website()**

La función **get_pathway_name()**: regresa los nombres de cada pathway y los almacena en el caché

Funciones de análisis de reacciones

La función **get_reaction_compounds()**: extrae compuestos de ecuaciones de reacciones bioquímicas

La función **check_reaction_equivalents()**: encuentra reacciones relacionadas a través de relaciones directas (RELATED_RN field) y participación compartida de compuestos

La función **_parse_pathways_from_get()**: parsea datos de pathway de las respuestas de la API

Función de integración global de la información de cada reacción

La función **get_reaction_info()**: unifica de la evaluación del caché, la validación de la existencia de cada reacción, la identificación de IDs synthetic/compound, los Intentos de múltiples estrategias de mapeo y la generación de un diccionario de resultados estructurado.

2. Funciones de procesamiento

La función **process_csv()** comprende el procesamiento inicial del flujo de trabajo iniciando por la validación de la estructura del archivo CSV de entrada e inicialización del mapeo con una tasa límite; pasando por el procesamiento de reacciones en batches con reporte del progreso, estadísticas de seguimiento y preservación del estado; hasta el manejo de casos especiales como reacciones sintéticas, compound IDs y Reacciones Verified-pero-unmapped; finalizando con la generación de CSV de salida

La función **generate_reports()** comprende la creación de un reporte de resumen con estadísticas, un reporte detallado de los not-found y una lista de casos especiales (compounds, synthetic reactions)

3. Ejecución principal

La función **main()** configura la interfaz de línea de comando con los argumentos de entradas y salidas requeridas, controles que limitan los procesos, configuración del archivo de estado y textos de ayuda o defectos

Ejemplo de flujo de trabajo del script

Para cada reaction ID:

Normaliza el formato del ID

Evalúa resultados previos en el caché

Verifica existencia en KEGG

Ejecuta múltiples estrategias de mapeo

Almacena resultados en metadata

Actualiza el estado y estadísticas de los resultados

El script **ReacToPath12.py** implementa manejo de errores a través de detección comprensiva de errores de la API y mecanismos de retorno al fallar las solicitudes. Por otro lado, el sistema

almacena datos en el caché sobre solicitudes de datos, relaciones de equivalence y nombres de cada pathway, mientras mantiene un estado persistente entre ejecuciones. Adicionalmente, para seguir las guías de KEGG API, el sistema instaura una tasa límite de solicitudes con un procesamiento de batch ajustable y retrasos automáticos entre solicitudes. Para verificar datos de forma comprensiva el script ejecuta verificaciones de múltiples orígenes, inicialmente usando REST API mientras se vuelve a buscar en la página web y descubrimiento de compuestos equivalentes de ser necesario. Finalmente, el script genera un reporte detallado de los resultados incluyendo archivos CSV, resúmenes estadísticos y reportes especializados de casos problemáticos.

Se ejecutó el comando **python3 ReacToPath12.py -i FenotiposMetabolicos.csv -o FenotiposMetabolicosReacToPath12.2.csv --batch-size 1 --delay 3.5** cuatro veces, pues la cuarta vez no encontró más IDs.

```
Progress: 100% | Found: 1967 | KEGG Verified: 7 | Via Equiv: 0 | Compound  
s: 0 | Synthetic: 1 | Not Found: 89
```

```
Processing complete!  
Results saved to FenotiposMetabolicosReacToPath12.2.csv  
Summary report saved to FenotiposMetabolicosReacToPath12.2_summary.txt  
Detailed not found report saved to FenotiposMetabolicosReacToPath12.2_not  
_found_detailed.csv
```

```
Progress: 100% | Found: 1969 | KEGG Verified: 5 | Via Equiv: 0 | Compound  
s: 0 | Synthetic: 1 | Not Found: 89
```

```
Processing complete!  
Results saved to FenotiposMetabolicosReacToPath12.3.csv  
Summary report saved to FenotiposMetabolicosReacToPath12.3_summary.txt  
Detailed not found report saved to FenotiposMetabolicosReacToPath12.3_not  
_found_detailed.csv
```

```
Progress: 100% | Found: 1971 | KEGG Verified: 3 | Via Equiv: 0 | Compound  
s: 0 | Synthetic: 1 | Not Found: 89
```

```
Processing complete!  
Results saved to FenotiposMetabolicosReacToPath12.5.csv  
Summary report saved to FenotiposMetabolicosReacToPath12.5_summary.txt  
Detailed not found report saved to FenotiposMetabolicosReacToPath12.5_not  
_found_detailed.csv
```

```
Progress: 100% | Found: 1971 | KEGG Verified: 3 | Via Equiv: 0 | Compound
s: 0 | Synthetic: 1 | Not Found: 89
```

Processing complete!

Results saved to FenotiposMetabolicosReactoPath12.6.csv

Summary report saved to FenotiposMetabolicosReactoPath12.6_summary.txt

Detailed not found report saved to FenotiposMetabolicosReactoPath12.6_not_found_detailed.csv

Puede que los IDs que inician con las letras “RK” o “RB” sean reacciones sintéticas al igual que la reacción final “Ex_Biomass[c]”. En este caso, las ecuaciones de cada ID que incumpla con el formato pueden revelar si son sintéticas según su función. En los archivos de fenotipo metabólico se pueden ver IDs que no cumplen con el formato de KEGG:

GNU nano 7.2		phenotypeComparisson.csv				
RK0001	0	0	0	NaN	NaN	NaN

GNU nano 7.2		phenotypeComparisson.csv			
RK0007	388.630403993449	0	388.630403993449	Inf	>

GNU nano 7.2			phenotypeComparisson.csv					
RBS01	10.1279872794395		0.349243385356913		9.7787438940826	28.999797001421	0.034483	0.9>
RE001	0	0	NaN	NaN	NaN			
RE002	1000	22.5628590025203		977.43714099748	44.3206244336455		0.0225628590025203	0.9>
RE003	0.00308149077014559		9.29766654674551e-05		0.00298851410467813		33.1426251377472	0.0>
RE004	0	0	NaN	NaN	NaN			
RE005	85.1440696082914		34.483	50.6610696082914		2.46916073451531	0.40499591056242	>
RE006	0	0	NaN	NaN	NaN			
RE007	-999.372432576405		-34.3618231350391		965.010609441366		29.0838011897377	0.0>
RE008	1000	34.3834635425069		965.616536457493		29.0837483188318	0.0343834635425069	>
RE009	1000	34.483	965.517	28.999797001421	0.034483		0.965517	
RE010	0	34.483	34.483	0	Inf	1		
RE011	0	0	NaN	NaN	NaN			
RE012	270.31922281585	34.483	235.83622281585	7.83920258724154		0.127563995045557	0.872436004	>
RE013	0	0	NaN	NaN	NaN			
RB001	-0	-0	0	NaN	NaN	NaN		
RB002	-1000	-22.5628590025203		977.437140997479		44.3206244336455	0.0225628590025203	>
RB003	-0.00308149077014559		-9.29766654674551e-05		0.00298851410467813		33.1426251377472	0.0>
RB004	-0	-0	0	NaN	NaN	NaN		
RB005	-85.1440696082914		-34.4830000000001		50.6610696082913		2.4691607345153	0.404995910>
RB006	3.46744855050929e-12		-3.16191517413245e-13		3.78364006792253e-12		-10.9662921348315	-0.>

2. Evaluar por parejas la correlación de Pearson entre los flujos por reacción de cada perfil de flujos metabólicos

La correlación de Pearson mide la magnitud y dirección de una relación lineal entre dos variables continuas a través de un coeficiente “r” que varía de -1 a 1, donde un valor positivo, indica una correlación positiva (al incrementar una variable, la otra también tiende a hacerlo), mientras que un valor negativo indica una correlación negativa (al incrementar una variable, la otra tiende a decrecer). Entre más cercano el valor a 1 o -1, más fuerte será la relación lineal

entre dos variables continuas. Por otro lado, un valor cercano a 0 sugiere la inexistencia de una correlación lineal.

Con el comando **nano ReactoPath12.py** creé el archivo de python para mapear los IDs y le añadí el siguiente algoritmo:

```
#!/usr/bin/env python3
```

```
"""
```

Script de análisis de correlación con verificación. Este script lee un archivo CSV, calcula la correlación de pearson entre columnas especificadas, verifica los cálculos con una implementación manual y genera scatter plots individuales por cada combinación. Uso:

```
python correlation_analysis6.py --input input.csv --output_prefix output
"""
```

```
import argparse
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import pearsonr
from itertools import combinations
```

```
def parse_arguments():
```

```
    """Parsear argumentos de la línea de comando."""
```

```
    parser = argparse.ArgumentParser(description='Calculate and visualize Pearson
correlations between specified columns.')
```

```
    parser.add_argument('--input', required=True, help='Input CSV file path')
```

```
    parser.add_argument('--output_prefix', required=True, help='Prefix for output files')
```

```
    return parser.parse_args()
```

```
def load_and_validate_data(file_path):
```

```
    """Cargar archivo CSV y validar columnas requeridas."""
```

```
    try:
```

```
        df = pd.read_csv(file_path)
```

```
        required_columns = ['FlorBlanca', 'Colombia', 'Clon39', 'Clon17']
```

```
        # Evaluar si todas las columnas requeridas existen
```

```
        missing_columns = [col for col in required_columns if col not in df.columns]
```

```
        if missing_columns:
```

```
            raise ValueError(f"Missing required columns: {missing_columns}")
```

```
        # Convertir columnas a número, convertir errores en NaN
```

```
        for col in required_columns:
```

```

        df[col] = pd.to_numeric(df[col], errors='coerce')

    return df[required_columns]

except Exception as e:
    raise ValueError(f"Error loading data: {str(e)}")

def manual_pearson(x, y):
    """Implementación de verificación manual de la correlación de Pearson"""
    n = len(x)
    sum_x = np.sum(x)
    sum_y = np.sum(y)
    sum_xy = np.sum(x * y)
    sum_x2 = np.sum(x**2)
    sum_y2 = np.sum(y**2)

    numerator = n * sum_xy - sum_x * sum_y
    denominator = np.sqrt((n * sum_x2 - sum_x**2) * (n * sum_y2 - sum_y**2))

    if denominator == 0:
        return 0
    return numerator / denominator

def calculate_pair_correlation(df, col1, col2):
    """Cálculo de la correlación de Pearson con verificación"""
    valid_idx = df[[col1, col2]].notna().all(axis=1)
    x = df.loc[valid_idx, col1].values
    y = df.loc[valid_idx, col2].values

    if len(x) < 2:
        return np.nan, np.nan, 0, np.nan

    # Cálculo de correlación de pearson usando scipy
    scipy_corr, scipy_pval = pearsonr(x, y)

    # Cálculo manual de correlación de pearson para verificación
    manual_corr = manual_pearson(x, y)

    return scipy_corr, scipy_pval, len(x), manual_corr

def visualize_pair_correlation(df, col1, col2, output_prefix):
    """Crear y guardar el scatter plot con información de verificación"""
    scipy_corr, scipy_pval, n, manual_corr = calculate_pair_correlation(df, col1, col2)

```

```

plt.figure(figsize=(10, 6))
sns.regplot(x=col1, y=col2, data=df, scatter_kws={'alpha':0.6})

# Añadir ambos cálculos de correlación al plot
title = (f'{col1} vs {col2}\n'
        f'Scipy r = {scipy_corr:.3f} (p = {scipy_pval:.3e})\n'
        f'Manual r = {manual_corr:.3f}\n'
        f'n = {n}')
plt.title(title)
plt.xlabel(col1)
plt.ylabel(col2)

# Guardar la figura
output_path = f'{output_prefix}_{col1}_vs_{col2}.png'
plt.savefig(output_path, dpi=300, bbox_inches='tight')
plt.close()

print(f"Scatter plot saved to {output_path}")
return scipy_corr, scipy_pval, manual_corr

def save_correlation_results(corr_results, output_prefix):
    """Guardar los resultados de la correlación en un archivo CSV."""
    results_df = pd.DataFrame(corr_results,
                              columns=['Variable1', 'Variable2',
                                       'Scipy_Correlation', 'P-value',
                                       'Manual_Correlation', 'N'])
    output_path = f'{output_prefix}_verified_correlations.csv'
    results_df.to_csv(output_path, index=False)
    print(f"Verified correlation results saved to {output_path}")

def main():
    try:
        args = parse_arguments()

        # Cargar y validar datos con diagnósticos adicionales
        print(f"Loading data from {args.input}...")
        data = load_and_validate_data(args.input)
        print(f"Data loaded successfully with {len(data)} rows.")

        print("\nData Summary Statistics:")
        print(data.describe())

        print("\nFirst 5 rows of data:")
        print(data.head())

```

```

# Generar pares únicos con todas las columnas
columns = data.columns
pairs = list(combinations(columns, 2))
print(f"\nGenerating plots and calculations for {len(pairs)} pairs:")

# Calcular y visualizar correlaciones para cada par de columnas
corr_results = []
for col1, col2 in pairs:
    print(f"\nProcessing pair: {col1} vs {col2}")
    scipy_corr, scipy_pval, manual_corr = visualize_pair_correlation(
        data, col1, col2, args.output_prefix)

    print(f"Scipy correlation: {scipy_corr:.6f}")
    print(f"Manual correlation: {manual_corr:.6f}")
    print(f"Absolute difference: {abs(scipy_corr - manual_corr):.6f}")

    n = data[[col1, col2]].notna().all(axis=1).sum()
    corr_results.append([col1, col2, scipy_corr, scipy_pval, manual_corr, n])

# Guarda todos los resultados de las correlaciones
save_correlation_results(corr_results, args.output_prefix)

print("\nAnalysis completed successfully.")

except Exception as e:
    print(f"Error: {str(e)}")
    exit(1)

if __name__ == "__main__":
    main()

```

Se ejecutó el comando **python3 correlation_analysis6.py --input FenotiposMetabolicosReacToPath12.6.csv --output_prefix Results12.6**

```
(base) gibbs@gibbs:~/FluxProcessing$ python3 correlation_analysis6.py --input FenotiposMetabolicosReacToPath
12.6.csv --output_prefix Results12.6
Loading data from FenotiposMetabolicosReacToPath12.6.csv...
Data loaded successfully with 2064 rows.

Data Summary Statistics:
      FlorBlanca      Colombia      Clon39      Clon17
count  2064.000000  2064.000000  2064.000000  2064.000000
mean   -1.210148   -3.179608   -2.566477   -1.881641
std     33.778392   88.751174   71.637083   52.521527
min    -70.292000  -184.689000  -149.075000  -109.296000
25%     0.000000    0.000000    0.000000    0.000000
50%     0.000000    0.000000    0.000000    0.000000
75%     0.000000    0.000000    0.000000    0.000000
max     70.292000   184.689000   149.075000   109.296000

First 5 rows of data:
      FlorBlanca      Colombia      Clon39      Clon17
0  -7.029200e+01  -1.846890e+02  -1.490750e+02  -1.092960e+02
1  -7.029200e+01  -1.846890e+02  -1.490750e+02  -1.092960e+02
2   4.653538e-30  -2.792123e-29   4.617183e-30  -9.307077e-30
3   5.015234e-02  1.317730e-01   1.063629e-01   7.798115e-02
4  -7.029200e+01  -1.846890e+02  -1.490750e+02  -1.092960e+02

Processing pair: FlorBlanca vs Colombia
Scatter plot saved to Results12.6_FlorBlanca_vs_Colombia.png
Scipy correlation: 1.000000
Manual correlation: 1.000000
Absolute difference: 0.000000

Processing pair: FlorBlanca vs Clon39
Scatter plot saved to Results12.6_FlorBlanca_vs_Clon39.png
Scipy correlation: 1.000000
Manual correlation: 1.000000
Absolute difference: 0.000000

Processing pair: FlorBlanca vs Clon17
Scatter plot saved to Results12.6_FlorBlanca_vs_Clon17.png
Scipy correlation: 1.000000
Manual correlation: 1.000000
Absolute difference: 0.000000

Processing pair: Colombia vs Clon39
Scatter plot saved to Results12.6_Colombia_vs_Clon39.png
Scipy correlation: 1.000000
Manual correlation: 1.000000
Absolute difference: 0.000000

Processing pair: Colombia vs Clon17
Scatter plot saved to Results12.6_Colombia_vs_Clon17.png
Scipy correlation: 1.000000
Manual correlation: 1.000000
Absolute difference: 0.000000

Processing pair: Clon39 vs Clon17
Scatter plot saved to Results12.6_Clon39_vs_Clon17.png
Scipy correlation: 1.000000
Manual correlation: 1.000000
Absolute difference: 0.000000
```

3. Evaluar la sumatoria absoluta o logarítmica (dividida el número de reacciones) entre los perfiles metabólicos en un heat map

Usando el comando **tmux attach-session -t 0** accedemos a la sesión **0** de tmux en la cual podemos ejecutar comandos directamente en el servidor, los cuales son inmunes a desconexiones entre la máquina local y el servidor. En la sesión de tmux para construir el script de mapeo se usó la herramienta nano, la cual ya estaba instalada en el servidor.

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux attach-session -t 0
[detached (from session 0)]
```

Con el comando **nano heatmap_analysis5.py** creé el archivo de python para Heatmap Generator for Absolute Log Summatory Divided by N y le añadí el siguiente algoritmo:

```
#!/usr/bin/env python3
```

```
"""
```


Generador de mapa de calor con evaluación de supuestos estadísticos. Incluye tests de normalidad y homogeneidad de varianzas.

```
"""
```

```
import argparse
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import pearsonr, spearmanr, kendalltau, shapiro, levene, bartlett,
probplot
import math
import warnings
from itertools import combinations
```

```
# Suprimir alertas de Shapiro para muestras > 5000
warnings.filterwarnings("ignore", category=UserWarning, message="p-value may not be
accurate")
```

```
def parse_arguments():
    """Parsear argumentos de la línea de comandos."""
    parser = argparse.ArgumentParser(description='Generate optimized heatmap with
statistical assumptions checking.')
    parser.add_argument('--input', required=True, help='Input CSV file path')
    parser.add_argument('--output_prefix', required=True, help='Prefix for output files')
    parser.add_argument('--metric', choices=['log_ratio', 'pearson', 'spearman', 'kendall',
'cosine'],
                        default='log_ratio', help='Comparison metric to use')
    parser.add_argument('--alpha', type=float, default=0.05,
                        help='Significance level for normality tests (default: 0.05)')
    return parser.parse_args()
```

```
def load_and_validate_data(file_path):
    """Cargar el archivo CSV y validar columnas requeridas."""
    try:
        df = pd.read_csv(file_path)
        required_columns = ['FlorBlanca', 'Colombia', 'Clon39', 'Clon17']

        missing_columns = [col for col in required_columns if col not in df.columns]
        if missing_columns:
            raise ValueError(f"Missing required columns: {missing_columns}")

        for col in required_columns:
            df[col] = pd.to_numeric(df[col], errors='coerce')
```

```

    return df[required_columns]

except Exception as e:
    raise ValueError(f"Error loading data: {str(e)}")

def check_normality(df, output_prefix, alpha=0.05):
    """Evaluar normalidad para cada columna usando Shapiro-Wilk test."""
    print("\nEvaluando normalidad de los datos (Shapiro-Wilk test):")
    normality_results = []

    plt.figure(figsize=(12, 8))
    for i, col in enumerate(df.columns, 1):
        data = df[col].dropna()
        if len(data) < 3:
            print(f" {col}: Insufficient data (n < 3)")
            continue

        # Test de Shapiro-Wilk
        stat, p = shapiro(data)
        is_normal = p > alpha

        # Gráfico Q-Q
        plt.subplot(2, 2, i)
        probplot(data, plot=plt)
        plt.title(f'Q-Q Plot - {col}\nShapiro p = {p:.4f}')

        normality_results.append({
            'Column': col,
            'n': len(data),
            'Shapiro_stat': stat,
            'Shapiro_p': p,
            'Is_normal': is_normal
        })

    plt.tight_layout()
    plt.savefig(f"{output_prefix}_qq_plots.png", dpi=300)
    plt.close()

    # Imprimir resultados
    for result in normality_results:
        print(f" {result['Column']}: n = {result['n']}, p = {result['Shapiro_p']:.4f} - {'Normal' if
result['Is_normal'] else 'No normal'}")

```

```

return normality_results

def check_homogeneity(df, output_prefix, alpha=0.05):
    """Evaluar homogeneidad de varianzas usando Levene y Bartlett tests."""
    print("\nEvaluando homogeneidad de varianzas:")

    # Preparar datos para el test
    data_to_test = [df[col].dropna().values for col in df.columns]

    # Test de Levene (robusto a no-normalidad)
    levene_stat, levene_p = levene(*data_to_test)
    levene_result = levene_p > alpha

    # Test de Bartlett (solo para datos normales)
    bartlett_stat, bartlett_p = bartlett(*data_to_test)
    bartlett_result = bartlett_p > alpha

    # Boxplot de comparación
    plt.figure(figsize=(10, 6))
    df_melted = df.melt(var_name='Group', value_name='Value')
    sns.boxplot(x='Group', y='Value', data=df_melted)
    plt.title('Distribución por grupo\nLevene p = {:.4f}, Bartlett p = {:.4f}'.format(levene_p,
    bartlett_p))
    plt.savefig(f"{output_prefix}_variance_comparison.png", dpi=300)
    plt.close()

    print(f" Levene test (robusto): p = {levene_p:.4f} - {'Varianzas iguales' if levene_result
    else 'Varianzas diferentes'}")
    print(f" Bartlett test (normal): p = {bartlett_p:.4f} - {'Varianzas iguales' if bartlett_result
    else 'Varianzas diferentes'}")

    return {
        'levene_stat': levene_stat,
        'levene_p': levene_p,
        'levene_result': levene_result,
        'bartlett_stat': bartlett_stat,
        'bartlett_p': bartlett_p,
        'bartlett_result': bartlett_result
    }

def calculate_log_ratio(df, col1, col2):
    """Calcular el log ratio por par de columnas."""
    valid_idx = df[[col1, col2]].notna().all(axis=1)
    x = df.loc[valid_idx, col1].values

```

```
y = df.loc[valid_idx, col2].values
```

```
if len(x) < 2:  
    return np.nan
```

```
with np.errstate(divide='ignore', invalid='ignore'):  
    ratio = np.sum(x) / np.sum(y)  
    if ratio <= 0:  
        return np.nan  
    return math.log10(ratio)
```

```
def calculate_correlation(df, col1, col2, method='pearson'):
```

```
    """Calcular correlación entre columnas."""
```

```
    valid_idx = df[[col1, col2]].notna().all(axis=1)
```

```
    x = df.loc[valid_idx, col1].values
```

```
    y = df.loc[valid_idx, col2].values
```

```
    if len(x) < 2:  
        return np.nan
```

```
    if method == 'pearson':  
        return pearsonr(x, y)[0]  
    elif method == 'spearman':  
        return spearmanr(x, y)[0]  
    elif method == 'kendall':  
        return kendalltau(x, y)[0]  
    return np.nan
```

```
def calculate_cosine_similarity(df, col1, col2):
```

```
    """Calcular la similitud de coseno entre columnas."""
```

```
    valid_idx = df[[col1, col2]].notna().all(axis=1)
```

```
    x = df.loc[valid_idx, col1].values
```

```
    y = df.loc[valid_idx, col2].values
```

```
    if len(x) < 2:  
        return np.nan
```

```
    dot_product = np.dot(x, y)  
    norm_x = np.linalg.norm(x)  
    norm_y = np.linalg.norm(y)
```

```
    if norm_x == 0 or norm_y == 0:  
        return np.nan
```

```
return dot_product / (norm_x * norm_y)
```

```
def generate_heatmap(df, output_prefix, metric='log_ratio'):
```

```
    """Genera y guarda el mapa de calor."""
```

```
    columns = df.columns
```

```
    n = len(columns)
```

```
    matrix = np.zeros((n, n))
```

```
    metric_names = {
```

```
        'log_ratio': 'Log10 Ratio ( $\sum x / \sum y$ )',
```

```
        'pearson': 'Pearson Correlation',
```

```
        'spearman': 'Spearman Correlation',
```

```
        'kendall': 'Kendall Tau Correlation',
```

```
        'cosine': 'Cosine Similarity'
```

```
    }
```

```
    # Calcular la métrica elegida para todas las parejas
```

```
    for i, col1 in enumerate(columns):
```

```
        for j, col2 in enumerate(columns):
```

```
            if metric == 'log_ratio':
```

```
                matrix[i, j] = calculate_log_ratio(df, col1, col2)
```

```
            elif metric in ['pearson', 'spearman', 'kendall']:
```

```
                matrix[i, j] = calculate_correlation(df, col1, col2, method=metric)
```

```
            elif metric == 'cosine':
```

```
                matrix[i, j] = calculate_cosine_similarity(df, col1, col2)
```

```
    # Crea un DataFrame para mejor etiquetado
```

```
    result_df = pd.DataFrame(matrix, index=columns, columns=columns)
```

```
    # Genera un mapa de calor
```

```
    plt.figure(figsize=(10, 8))
```

```
    sns.heatmap(
```

```
        result_df,
```

```
        annot=True,
```

```
        fmt=".3f",
```

```
        cmap='coolwarm' if metric != 'log_ratio' else 'viridis',
```

```
        center=0 if metric != 'log_ratio' else None,
```

```
        square=True,
```

```
        linewidths=.5,
```

```
        cbar_kws={'label': metric_names.get(metric, 'Metric')},
```

```
        annot_kws={"size": 10}
```

```
    )
```

```
    plt.title(f'Heatmap of {metric_names.get(metric, "Column Comparisons")}')  
    plt.tight_layout()
```

```

# Guardar las salidas
output_path = f"{output_prefix}_{metric}_heatmap.png"
plt.savefig(output_path, dpi=300, bbox_inches='tight')
plt.close()

result_df.to_csv(f"{output_prefix}_{metric}_heatmap_values.csv")

print(f"Heatmap saved to {output_path}")
print(f"Matrix values saved to {output_prefix}_{metric}_heatmap_values.csv")

def main():
    try:
        args = parse_arguments()

        print(f"Loading data from {args.input}...")
        data = load_and_validate_data(args.input)
        print(f>Data loaded successfully with {len(data)} rows.")

        # Evaluación de supuestos estadísticos
        normality_results = check_normality(data, args.output_prefix, args.alpha)
        homogeneity_results = check_homogeneity(data, args.output_prefix, args.alpha)

        # Recomendación de métrica basada en los resultados
        all_normal = all(result['is_normal'] for result in normality_results)
        equal_variances = homogeneity_results['levene_result']

        print("\nRecomendación estadística:")
        if all_normal and equal_variances:
            print(" Los datos son normales y tienen varianzas iguales.")
            print(" Puedes usar tests paramétricos (Pearson).")
        else:
            print(" Los datos NO cumplen los supuestos para tests paramétricos.")
            print(" Considera usar métodos no paramétricos (Spearman/Kendall).")

        print(f"\nGenerating {args.metric} comparison heatmap...")
        generate_heatmap(data, args.output_prefix, args.metric)

        print("\nAnalysis completed successfully.")

    except Exception as e:
        print(f"Error: {str(e)}")
        exit(1)

```

```
if __name__ == "__main__":  
    main()
```

```
(base) gibbs@gibbs:~/FluxProcessing$ python heatmap_analysis5.py --help  
usage: heatmap_analysis5.py [-h] --input INPUT --output_prefix OUTPUT_PREFIX  
                           [--metric {log_ratio,pearson,spearman,kendall,cosine}] [--alpha ALPHA]  
  
Generate optimized heatmap with statistical assumptions checking.  
  
options:  
  -h, --help            show this help message and exit  
  --input INPUT          Input CSV file path  
  --output_prefix OUTPUT_PREFIX  
                        Prefix for output files  
  --metric {log_ratio,pearson,spearman,kendall,cosine}  
                        Comparison metric to use  
  --alpha ALPHA          Significance level for normality tests (default: 0.05)
```

Manejo de entrada: toma la ruta del archivo CSV y el prefijo de los archivos de salida como argumentos de la línea de comando. Luego, en el archivo de entrada busca específicamente las columnas: FlorBlanca, Colombia, Clon39, Clon17. Transforma los datos faltantes en NaN.

Calcula para cada par de columnas (incluyendo comparaciones entre las mismas columnas en diagonal): Calcula la suma de valores absolutos ($x + y$) para evadir logaritmos negativos. Luego toma el logaritmo natural de la suma como valor absoluto y lo divide por el número de observaciones válidas (n).

Como salida genera la visualización de un heatmap con valores anotados en cada celda, guarda ambos, la imagen del heatmap (PNG) y la matriz cruda de valores (CSV). Adicionalmente usa viridis colormap para mejorar la distinción en la visualización.

El manejo de errores implica la evaluación de columnas faltantes, cálculos inválidos de logaritmos y la generación de mensajes informativos de error

Ejecuté el comando **python heatmap_analysis3.py --input FenotiposMetabolicosReacToPath12.6.csv --output_prefix results12.6.3**

```
(base) gibbs@gibbs:~/FluxProcessing$ python heatmap_analysis3.py --input FenotiposMetabolicosReacToPath12.6.csv --output_prefix results12.6.3  
Loading data from FenotiposMetabolicosReacToPath12.6.csv...  
Data loaded successfully with 2064 rows.  
  
Generating log_ratio comparison heatmap...  
Heatmap saved to results12.6.3_log_ratio_heatmap.png  
Matrix values saved to results12.6.3_log_ratio_heatmap_values.csv  
  
Analysis completed successfully.
```

```
(base) gibbs@gibbs:~/FluxProcessing$ python heatmap_analysis5.py --input FenotiposMetabolicosReacToPath12.6.csv --output_p
refix results13.1 --metric cosine
Loading data from FenotiposMetabolicosReacToPath12.6.csv...
Data loaded successfully with 2064 rows.

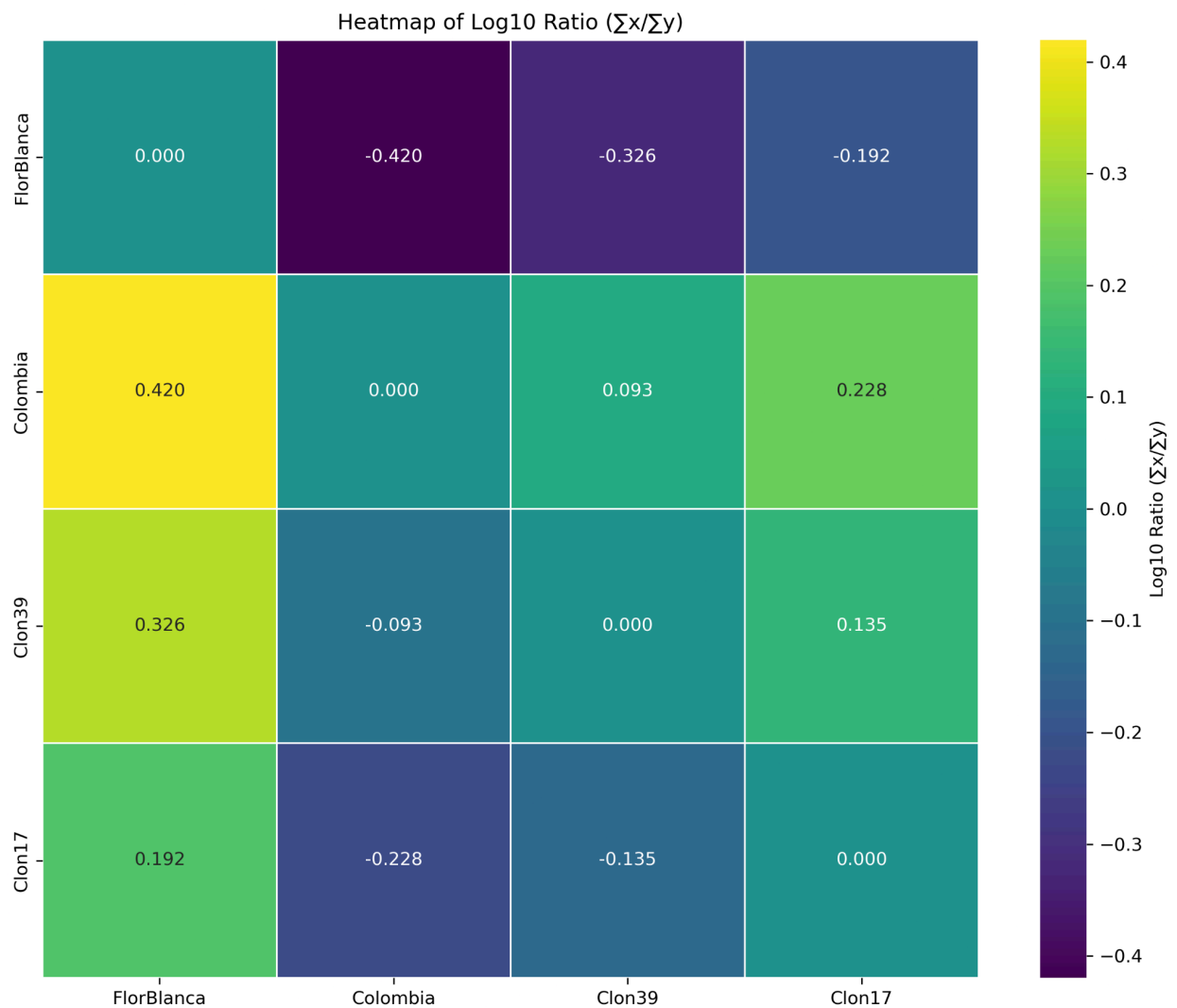
Evaluando normalidad de los datos (Shapiro-Wilk test):
FlorBlanca: n = 2064, p = 0.0000 - No normal
Colombia: n = 2064, p = 0.0000 - No normal
Clon39: n = 2064, p = 0.0000 - No normal
Clon17: n = 2064, p = 0.0000 - No normal

Evaluando homogeneidad de varianzas:
Levene test (robusto): p = 0.0000 - Varianzas diferentes
Bartlett test (normal): p = 0.0000 - Varianzas diferentes

Recomendación estadística:
Los datos NO cumplen los supuestos para tests paramétricos.
Considera usar métodos no paramétricos (Spearman/Kendall).

Generating cosine comparison heatmap...
Heatmap saved to results13.1_cosine_heatmap.png
Matrix values saved to results13.1_cosine_heatmap_values.csv

Analysis completed successfully.
```



results12.6_heatmap

Una distribución normal o de campana de Gauss es una distribución de probabilidad simétrica en forma de campana, con la mayoría de datos agrupados alrededor del valor promedio.

La homogeneidad de varianza o homocedasticidad, es un supuesto estadístico que propone que la varianza o distribución de datos entre grupos comparados es similar. Es decir que la variabilidad en cada grupo analizado es similar.

Script de submódulos **filter_kegg_pathways9.py**:

```
import argparse
import csv

def is_potato_pathway(pathway_name):
    """Check if a pathway is specific to Solanum tuberosum (potato) or general plants."""
    # List of pathway names to exclude (microbial, animal-specific, etc.)
    exclude_keywords = [
        'microbial', 'bacterial', 'animal', 'human', 'mammal',
        'virus', 'fungal', 'yeast', 'prokaryotic', 'diverse environments'
    ]

    # Check if any exclude keyword is in the pathway name (case-insensitive)
    for keyword in exclude_keywords:
        if keyword.lower() in pathway_name.lower():
            return False

    # If none of the exclude keywords are found, assume it's a plant/potato pathway
    return True

def process_pathways(kegg_ids, pathway_names):
    """Process pathway pairs, flagging non-potato ones."""
    processed_kegg_ids = []
    processed_pathway_names = []

    for kegg_id, pathway in zip(kegg_ids, pathway_names):
        if is_potato_pathway(pathway):
            processed_kegg_ids.append(kegg_id)
            processed_pathway_names.append(pathway)
        else:
```

```

        processed_kegg_ids.append(f"{kegg_id} (unknown for S. tuberosum)")
        processed_pathway_names.append(f"{pathway} (unknown for S. tuberosum)")

    return processed_kegg_ids, processed_pathway_names

def filter_kegg_pathways(input_file, output_file):
    """Process KEGG pathways, flagging non-potato ones while keeping all entries."""
    with open(input_file, 'r') as infile, open(output_file, 'w', newline='') as outfile:
        reader = csv.DictReader(infile)
        fieldnames = reader.fieldnames
        writer = csv.DictWriter(outfile, fieldnames=fieldnames)
        writer.writeheader()

        for row in reader:
            if not row['Map_KEGG_IDs'] or not row['Pathway_Names']:
                writer.writerow(row)
                continue

            # Split the semicolon-separated lists
            kegg_ids = [id.strip() for id in row['Map_KEGG_IDs'].split(';')]
            pathway_names = [name.strip() for name in row['Pathway_Names'].split(';')]

            # Process all pathway pairs
            processed_kegg_ids, processed_pathway_names = process_pathways(kegg_ids,
pathway_names)

            # Update the row with processed data
            row['Map_KEGG_IDs'] = '; '.join(processed_kegg_ids)
            row['Pathway_Names'] = '; '.join(processed_pathway_names)
            writer.writerow(row)

def main():
    parser = argparse.ArgumentParser(
        description='Process KEGG pathways, flagging non-Solanum tuberosum ones.',
        formatter_class=argparse.ArgumentDefaultsHelpFormatter
    )
    parser.add_argument('-i', '--input', required=True,
                        help='Input CSV file containing KEGG pathway data')
    parser.add_argument('-o', '--output', required=True,
                        help='Output CSV file with processed KEGG pathways')

    args = parser.parse_args()
    filter_kegg_pathways(args.input, args.output)
    print(f"Processed data saved to {args.output}")

```

```
if __name__ == '__main__':  
    main()
```

```
python3 filter_kegg_pathways9.py -i FenotiposMetabolicosReacToPath12.6.csv -o  
Filtrados.9.FenotiposMetabolicosReacToPath12.6.csv
```

```
python3                                expand_kegg_pathways.py                -i  
Filtrados.9.FenotiposMetabolicosReacToPath12.6.csv                        -o  
Expanded.Filtrados.9.FenotiposMetabolicosReacToPath12.6.csv
```

```
import argparse  
import csv
```

```
def expand_kegg_pathways(input_file, output_file):  
    """
```

Expand CSV rows by creating separate entries for each Map_KEGG_IDs and Pathway_Names pair.

Maintains all other column values for each expanded row.

```
    """
```

```
    with open(input_file, 'r') as infile, open(output_file, 'w', newline='') as outfile:
```

```
        reader = csv.DictReader(infile)
```

```
        fieldnames = reader.fieldnames
```

```
        writer = csv.DictWriter(outfile, fieldnames=fieldnames)
```

```
        writer.writeheader()
```

```
    for row in reader:
```

```
        if not row['Map_KEGG_IDs'] or not row['Pathway_Names']:
```

```
            # Write the row as-is if there are no pathways
```

```
            writer.writerow(row)
```

```
            continue
```

```
        # Split the semicolon-separated lists
```

```
        kegg_ids = [id.strip() for id in row['Map_KEGG_IDs'].split(';')]
```

```
        pathway_names = [name.strip() for name in row['Pathway_Names'].split(';')]
```

```
        # Check if we have matching numbers of IDs and pathway names
```

```
        if len(kegg_ids) != len(pathway_names):
```

```
            print(f"Warning: Mismatched counts in row {row['RxnID']} - {len(kegg_ids)} IDs vs  
{len(pathway_names)} pathways")
```

```

        continue

    # Create a new row for each KEGG ID/pathway pair
    for kegg_id, pathway in zip(kegg_ids, pathway_names):
        new_row = row.copy()
        new_row['Map_KEGG_IDs'] = kegg_id
        new_row['Pathway_Names'] = pathway
        writer.writerow(new_row)

def main():
    parser = argparse.ArgumentParser(
        description='Expand CSV rows by creating separate entries for each KEGG pathway.',
        formatter_class=argparse.ArgumentDefaultsHelpFormatter
    )
    parser.add_argument('-i', '--input', required=True,
                        help='Input CSV file with combined KEGG pathways')
    parser.add_argument('-o', '--output', required=True,
                        help='Output CSV file with expanded KEGG pathways')

    args = parser.parse_args()
    expand_kegg_pathways(args.input, args.output)
    print(f"Expanded data saved to {args.output}")

if __name__ == '__main__':
    main()

```

```

python3                                expand_kegg_pathways.py                -i
Filtrados.9.FenotiposMetabolicosReacToPath12.6.csv                -o
Expanded.Filtrados.9.FenotiposMetabolicosReacToPath12.6.csv

```

Comandos adicionales

A. La herramienta conda se puede descargar con el comando **wget** <https://www.anaconda.com/download/success> el cual descarga un archivo ejecutable bash (.sh) capaz de instalar la herramienta al ser ejecutado. Posterior a la instalación se debe reiniciar la consola o salir y volver a entrar al servidor, pues el programa lo requiere para luego ser activado con el comando **conda activate**

```

gibbs@gibbs:~/tools$ wget https://repo.anaconda.com/archive/Anaconda3-2024.10-1-Linux-x86_64.sh
--2025-04-09 20:24:46-- https://repo.anaconda.com/archive/Anaconda3-2024.10-1-Linux-x86_64.sh
Resolving repo.anaconda.com (repo.anaconda.com)... 104.16.32.241, 104.16.191.158, 2606:4700::6810:20f1, ...
Connecting to repo.anaconda.com (repo.anaconda.com)|104.16.32.241|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1102495056 (1.0G) [application/octet-stream]
Saving to: 'Anaconda3-2024.10-1-Linux-x86_64.sh'

64.sh                20%[=>] 218.20M  27.9MB/s  eta 31s

Please answer 'yes' or 'no':
>>> yes

Anaconda3 will now be installed into this location:
/home/gibbs/anaconda3

- Press ENTER to confirm the location
- Press CTRL-C to abort the installation
- Or specify a different location below

[/home/gibbs/anaconda3] >>>
PREFIX=/home/gibbs/anaconda3
Unpacking payload ...

```

```

gibbs@gibbs:~$ conda activate
(base) gibbs@gibbs:~$

```

B. La herramienta tmux se puede descargar e instalar usando los comandos **sudo apt update** seguido del comando **sudo apt install tmux**. Posterior a la instalación se puede activar la herramienta con el comando **tmux** y se desactiva con la combinación de teclas **CTRL+b d**. Si se quiere reabrir una sesión de tmux se puede usar el comando **tmux ls** para listar las sesiones de tmux existentes y luego usar el comando **tmux attach-session -t 0** donde lo que sigue después de la **-t** es el nombre de la sesión existente a la que se quiere acceder. Si uno quiere subir o bajar con las flechas del teclado en la sesión de tmux, primero se debe usar la combinación de teclas **ctrl+b** seguida de **alt+(** (eso en el caso de ubuntu), para salir de la modalidad de movimiento se oprime la tecla **q**.

```

(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ sudo apt update
[sudo] password for gibbs:

```

```

(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ sudo apt install tmux
Reading package lists... Done

```

```

(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux
[detached (from session 0)]
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$

```

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$

[0] 0:bash* "gibbs" 21:19 09-Apr-25

(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux ls
0: 1 windows (created Wed Apr 9 21:19:19 2025)

(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux attach-session -t 0
[detached (from session 0)]
```

Por otro lado, se pueden crear sesiones tmux con nombres particulares usando el comando **tmux new -s session_name** donde lo que sigue después de la **-s** es el nombre que se quiere dar a la sesión. Con el comando **tmux ls** se pueden confirmar las sesiones de tmux existentes entre las que ya se encuentra **session_name** y **0**.

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux new -s session_name
[detached (from session session_name)]

(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux ls
0: 1 windows (created Wed Apr 9 21:19:19 2025)
session_name: 1 windows (created Wed Apr 9 21:36:42 2025)
```

Aunque si se quiere borrar una sesión se puede usar el comando **tmux kill-session -t session_name** donde **session_name** es el nombre de la sesión a borrar. Con el comando **tmux ls** se pueden confirmar las sesiones de tmux existentes entre las que ya no se encuentra **session_name**.

```
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux kill-session -t session_name
(base) gibbs@gibbs:~/analysis2.0/QualityControl/RawDataQC$ tmux ls
0: 1 windows (created Wed Apr 9 21:19:19 2025)
```

C. La herramienta **diff** permite ver diferencias entre archivos con comandos como **diff [option] file1 file2 d**